

23 USING THE GEOMETRY IN A RAY-TRACING APPLICATION

In this chapter, we put the algebra to work by using it to implement a ray tracer in the C++ programming language. A ray tracer is a program that takes as input the description of a scene (including light sources, models, and camera) and produces an image of that scene as seen from the camera. We show how the conformal model of Euclidean geometry is very useful for the specification and computation of the basic operations. All geometrical elements occurring in this problem have a natural representation in that model.

When we really try to optimize for the speed of the ray tracer, we argue that one may want to deviate from the natural all-purpose representation to a more dedicated choice, which anticipates the typical ray tracing operations that are going to be performed on the data. That is, we trade some of the elegance of the conformal model for added efficiency. Even then, each of these choices can be described and compared within the conformal model, which moreover always supplies the necessary transformations in its versor representation. The resulting compromise between algebraic elegance and practical performance is probably typical of many applications.

In the process, we write a graphical user interface to manipulate objects in a viewport, as well as the camera that produces the view. This is another application in which the conformal model shows its power by giving directly executable expressions connecting

mouse coordinates to the simulated motions. Here practical performance and algebraic elegance coincide in a satisfying manner.

23.1 RAY-TRACING BASICS

When you view a scene in the real world, rays originating from light sources interact with objects, picking up their colors before they reach your eye to form an image. In classical ray tracing, this process is reversed: starting from a pixel of the image, a ray is cast through the optical center of the camera and traced into the world. It encounters objects, reflects, refracts, and ultimately may reach one or more light sources. The product of all these interactions with the material and spectral properties of the objects and light sources determines the color, shading, and intensity that need to be rendered in the original pixel. This is called the *shading computation*. To do this physically exactly is impossible or slow, so many convenient shortcuts and simplifications have been introduced. Often, performing the shading computation requires new rays to be spawned and traced. Among those are the *shadow rays* that are used to check whether a direct path exists between some intersection point and a light source. New rays must also be spawned if the material at the intersection point is reflective or if it is transparent (which requires a new ray to be created after refraction, according to Snell's law).

From the algebraic point of view, the representation of rays and their intersections with objects is central to a ray tracer. The possibilities the conformal model offers for these elements will be carefully considered in their advantages and disadvantages, both algebraically and implementationally. Most of the time, algebraic elegance and computational efficiency go hand-in-hand, but sometimes we will be forced to choose, and then we will opt for efficiency. Yet we will always remain within the framework of the conformal model and benefit fully from its capability of specifying all operations in terms of the geometrical elements rather than their coordinates.

As we have seen in the previous chapter, using geometric algebra does not need to go at the expense of efficiency. A good code generator can convert the CGA specifications to executable code. (We will sometimes use CGA as the abbreviation for conformal geometric algebra in this chapter.) The code generator should certainly allow the creation of new types for specific conformal primitives (e.g., points, lines). But a good package goes further than merely creating the data structures. With a proper implementation, some seemingly expensive algebraic operations can be surprisingly cheap. For instance, to extract the Euclidean direction $\mathbf{u}$ from a line L , we can write $\mathbf{u} = (\infty \wedge o) \rfloor L$. In an efficient geometric algebra package, this computation requires very little effort, since it can be simplified to just assembling the coefficients L has on the basis blades $o \wedge \mathbf{e}_1 \wedge \infty$, $o \wedge \mathbf{e}_2 \wedge \infty$, and $o \wedge \mathbf{e}_3 \wedge \infty$ into a vector with basis $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{e}_3$. Dualization with respect to basis elements, in particular the pseudoscalar, can also be implemented by coordinate manipulation rather than true inner products. Such implementation considerations will affect our representational choices in the ray tracer considerably.

We will implement a complete but limited ray tracer. It includes a simple modeler with OpenGL visualization, in which one can interactively create the scene to be rendered. It can render only still frames (no animations). It can render only polygonal meshes (no curved surfaces). No kinematic chains are possible. It has support for texturing and bump mapping. Shading is adopted from the standard OpenGL shading model (ambient, diffuse, specular; see [58], Chapter 5). It supports reflection and refraction. We apply no optimizations to improve ray-model intersection test efficiency except simple bounding spheres and BSP trees.

The equations in this chapter are directly shown as C++ code, with occasional reference to their occurrence in the earlier chapters. Since the code is usually close to the mathematical notation, it should be easily readable even for non-C++ literate. The only catches are:

- The “.” denotes not the inner product, but access to a member variable in C++ .
- Since the “.” is already taken, we use the $\ll$ symbol to denote the inner product.
- We write n_i for ∞ and n_o for o .

The code examples we show are sometimes polished or slightly changed for presentation. The full, unedited source code can be inspected at <http://www.geometricalgebra.net>. We use *Gaigen 2* as our conformal model implementation in this chapter. It is not required to have read the programming examples in Parts I and II to understand this chapter. (The ray-tracer code is not built on top of the GA sandbox source code package because the ray tracer was created before we had that available.)

23.2 THE RAY-TRACING ALGORITHM

We present the basic outline of the ray-tracing algorithm to define our terms and geometrical subproblems.

To render an image, the ray tracer spawns rays through the optical center of the camera position and each pixel of the image. (For antialiasing, multiple rays per pixel can be spawned, each with a small offset.)

For each spawned ray we want to find the closest intersection with the surface of a model. A quick intersection test is performed by first checking the intersection with a bounding sphere that encloses the entire model. If that test turns out positive, a more complex intersection test is done. The polygons of the models are stored in binary space partition (BSP) trees. We descend down the BSP tree until we discover what polygons the ray intersects. We pick the intersection that was the closest to the start of the ray and return it as the result of the intersection test.

Once the closest intersection point is known, we query the appropriate polygon to supply information about the material at the intersection point. Material properties include

color, reflectivity, transparency, refractive index, and surface attitude (classically called surface normal, which is of course the dual of the direction 2-blade).

Using the material properties and light source information, local shading computations are made. The shading equations that we use are the same as those employed for the fixed function pipeline of OpenGL, except that we do shadow checks: for a light source to illuminate the intersection point, there must be a clear path between them. So a *shadow ray* is spawned from the intersection point towards each of the light sources to check the path. If the shadow ray cannot reach a light source unobstructed, that light source does not contribute to diffuse and specular lighting for the intersection point. Note that transparent objects are also treated as obstructions, because refraction causes our shadow ray to bend away from the light source, and we cannot easily compensate for that.

The outcome of the shading computations contributes to the final color of the pixel from which the ray originated. If the material is reflective or refractive, new rays are spawned appropriately. The colors returned by these rays also contribute to the final color of the pixel that the ray originated from.

In the next three sections we discuss the low-level geometric details of each part of the ray tracer. We start with the representation and interactive modeling of the scene, followed by the tracing of the rays, and end with shading computations.

23.3 REPRESENTING MESHES

To represent the shape of objects in the scene we need to model them. We will use a triangular mesh, consisting of a set of vertices and triangular faces, bounded by edges. Figure 23.1 shows an example of such a mesh in both solid and wireframe rendering.



Figure 23.1: A polygonal mesh rendered in solid and with its wireframe model superimposed.

Meshes are read from `.rtm` files, which describes the faces and vertices of the mesh. The description of a face is simply a list of indices of the three vertices. The vertex description is more involved, specifying position, local surface attitude, and texture coordinates. Finally, the mesh file contains material properties such as reflectivity, transparency, and optional texture information.

Vertices

Vertices are stored in the `vertex` class. Vertices naturally require a 3-D position $\mathbf{x}$. We choose to use a normalized conformal point (of the form $o + \mathbf{x} + \frac{1}{2}\mathbf{x}^2\infty$) to represent the position of a vertex. Such a normalized conformal point requires four coordinates for storage (the constant o does not have to be stored). Alternatively, we could have used a flat point (of the form $(o + \mathbf{x}) \wedge \infty$) that would require one less coordinate, but this would be annoying in the algebra of the spanning operation (in which the plane determined by three points is the outer product of three regular points with infinity, not flat points) and during barycentric interpolation (see equation (11.19)).

To initialize the points, we read the Euclidean x , y , and z coordinates from the `.rtm` and construct a point from them:

```
normalizedPoint pt = cgaPoint(x * e1 + y * e2 + z * e3);
```

`cgaPoint()` is a function that constructs a point that looks like the familiar (13.3):

```
// Returns a point at the location specified by 3D Euclidean vector p
normalizedPoint cgaPoint(const vectorE3GA &p)
{
    return p + no + 0.5 * norm_e2(p) * ni;
}
```

The `norm_e2()` function computes the squared Euclidean norm (i.e., the sum of the squares of all coordinates).

It is common practice to approximate the look of a smooth surface by interpolating the local surface attitude at the vertices across each faces. This affects only the shading computations and does not change the actual shape of the model (hence the contours of a model show that it is actually a polygonal mesh). The mesh files specify the *normals* for each vertex. We construct a free vector from these coordinates (to refresh your memory on the kinds of vectors in the conformal model, review Section 15.4). This free vector is then dualized into a free 2-blade that is used to represent the surface 2-direction (which we call its *attitude*). Conceptually, attitude and normal represent the same geometrical feature, either by a 2-blade or by a perpendicular vector.

```
// nx, ny and nz are floats.
// They represent the classical surface normal as an attitude.
freeBivector att = dual((nx * e1 + ny * e2 + nz * e3) ^ ni);
```

Each vertex also has an associated 2-D point that is used for *texture mapping*, which is a common way of coloring the surface of a 3-D model by wrapping a 2-D image

around it. In computer graphics, texture mapping is also used to apply bumps to the surface of a model (bump mapping, or displacement mapping), to specify the surface transparency, and so on. Because we won't be doing a lot of geometry on the texture coordinates, we use *normalized flat 2-D points* to represent them (this requires only two coordinates). That is effectively equivalent to representing them by homogeneous coordinates (see Section 13.3.2). Note that these points live in a different space than the rest of our geometry, namely in the 2-D carrier space of the texture image.

So putting it all together, the storage part of the `vertex` class looks like

```
class vertex {
    normalizedPoint pt;           // position of the vertex
    freeBivector att;             // local surface attitude
    normalizedFlatPoint2d texPoint; // position in 2D texture image(s)
};
```

Representing Vertices with Tangent 2-Blades

An alternative way to represent the position and the local surface direction of a vertex is to use a tangent 2-blade. We can combine point p and attitude A into one vertex primitive with their combined properties and the clearer semantics of a tangent 2-blade: $V = p \rfloor (p \wedge A \wedge \infty)$, see (15.1).

However, there are some efficiency concerns with this tangent 2-blade representation, for though its attitude can be extracted for free, its position cannot. The attitude of a tangent 2-blade V is the free 2-blade algebraically given by $-(\infty \rfloor V) \wedge \infty$, and this can be extracted by taking the $\phi \wedge e_i \wedge e_j$ coordinates of V and making them into $e_i \wedge e_j \wedge \infty$ -coordinates of an attitude element $A \wedge \infty$. But the position of a tangent 2-blade cannot be extracted in this manner from V . A method to get the flat point at the location of V is $(\infty \rfloor V) \rfloor (V \wedge \infty)$, which you may recognize as the `meet` of the line perpendicularly through V with the plane that contains V . On the coordinate level, implementing this equation involves some multiplication and addition rather than mere coordinate transfer. Since the position of the vertices is required often during rendering to do interpolation, the cost of its extraction made us decide against using the tangent 2-blade representation of vertices for the ray tracer. This is an example where a more algebraically elegant possibility is overruled by efficiency considerations.

Faces

The `face` class describes the faces of the polygonal model. The main data in this class are the indices of the vertices that make up the face. These indices are read from the `.rtm` file, stored in an integer array `vtxIdx`, and accessed through a function `vtx()` that returns the vertex.

The ray tracer then precomputes some blades to speed up later computations using standard conformal operations:

- The plane that contains the face:

```
// wedge together the position of vertex 0, 1, 2 and infinity
p1 = vtx(0).pt ^ vtx(1).pt ^ vtx(2).pt ^ ni;
```

We also check whether the face actually spans surface area:

```
if (norm_e2(p1) == 0.0) // -> no surface area
```

- The three conformal lines along the three edges of the polygon:

```
// each edge line is its (start vertex)^(end vertex)^infinity
edgeLine[0] = vtx(0).pt ^ vtx(1).pt ^ ni;
edgeLine[1] = vtx(1).pt ^ vtx(2).pt ^ ni;
edgeLine[2] = vtx(2).pt ^ vtx(0).pt ^ ni;
```

The edge lines are used to compute whether a specific line intersects the face, and they come in handy to compute the direction of edges. For example,

```
((ni ^ no) << edgeLine[0]) ^ ni}
```

is the free vector along the direction of edge 0. With these details, the storage part of the face class looks like:

```
class face {
    // indices of the three vertices
    int vtxIdx[3];

    // lines along the edges of the face
    line edgeLine[3];

    // the plane of the face
    plane p1;
}
```

Computing the Bounding Sphere

For each mesh, a bounding sphere is computed that contains all the vertices of the mesh. As we mentioned before, this is a common trick to speed up intersection computations: when we need to test for the intersection of some primitive with a model, we do not do a detailed intersection test straight away, but instead first check if it intersects the bounding sphere. If so, then we proceed with the detailed intersection test, otherwise we can report that no intersection was found. The bounding sphere does not need to be tight (although the tighter the better). For simplicity we compute a sphere of which the center is at half the extent of the data set in the directions of the coordinate axes, and with an appropriate radius. This sphere is stored as a regular conformal sphere.

```
/*
Compute the center of the bounding sphere:
- create three orthogonal planes through the origin:
- compute minimal and maximal signed distance of each
vertex to these planes
*/
```

```

dualPlane pld[3] = {
    dual(no ^ e2 ^ e3 ^ ni),
    dual(no ^ e3 ^ e1 ^ ni),
    dual(no ^ e1 ^ e2 ^ ni)
};
mv::Float minDist[3] = {1e10, 1e10, 1e10};
mv::Float maxDist[3] = {-1e10, -1e10, -1e10};
for (int i = 0; i < vertices.size(); i++) {
    for (int j = 0; j < 3; j++) {
        minDist[j] = min(Float(pld[j] << vertices[i].pt)), minDist[j]);
        maxDist[j] = max(Float(pld[j] << vertices[i].pt)), maxDist[j]);
    }
}

/*
Compute the center 'c' of the bounding sphere.
*/
normalizedPoint c(cgaPoint(
    0.5 * (minDist[0] + maxDist[0]) * pld[0] +
    0.5 * (minDist[1] + maxDist[1]) * pld[1] +
    0.5 * (minDist[2] + maxDist[2]) * pld[2]));

/*
Compute the required radius 'r' (actually, -radius^2/2) of the
bounding sphere:
*/
mv::Float r = 0.0;
for (int i = 0; i < vertices.size(); i++)
    r = min(vertices[i].pt << center, r)

/*
Construct the bounding dual sphere:
*/
boundingSphere = c + r * ni;

```

Constructing the BSP Tree

To do a detailed intersection test of a model with, say, a line, we could simply check the intersection of the line with every face of the model. For complex models with many faces, this can become computationally expensive.

To speed up the intersection test, we apply a standard technique that is similar to bounding spheres but employed recursively. First we search a plane that has about half of the vertices of the model at its back side and the other half of the vertices in front. We sort the vertices according to the side of the plane on which they lie, and then recurse: for each of the halves of the vertex sets, we search for another plane that splits these halves in half. And so on, until we have partitioned the space into chunks that each contain a reasonable number of vertices. This is called a *binary space partition tree* (BSP tree). The intersection test of a line with such a BSP tree is discussed in Section 23.5.3.

During the construction of the BSP tree, the main geometric computation is the selection of the plane to partition the space. We use the following approach: for the subdivision planes, we cycle, modulo 3, through translated versions of the three orthogonal planes $o \wedge e_2 \wedge e_3 \wedge \infty$, $o \wedge e_3 \wedge e_1 \wedge \infty$, and $o \wedge e_1 \wedge e_2 \wedge \infty$. The remaining difficulty is to compute the translation that halves the vertex set in the chosen direction. This is done by applying the Quicksort algorithm. All of these computations are just standard techniques cast into a conformal notation.

```

/*
Get the 'base plane' we are going to use for partitioning the space:
The integer 'basePlaneIdx' tells us what plane to use:
*/
dualPlane dualBasePlane;
if (basePlaneIdx == 0) dualBasePlane = dual(no ^ e2 ^ e3 ^ ni);
else if (basePlaneIdx == 1) dualBasePlane = dual(no ^ e3 ^ e1 ^ ni);
else dualBasePlane = dual(no ^ e1 ^ e2 ^ ni);

/*
Compute the signed distance of each vertex to the plane.
This distance is stored inside the vertices,
so we can use it to sort them:
*/
for (i = 0; i < nbVertices; i++)
    vtx(i).setSignedDistance(
        dualBasePlane << getVertex(vertex[i].idx).pt);

/*
Use quicksort to sort the vertices.
The function 'constructBSPTreeSort()' compares the
signed_distance fields of the vertices.
*/
qsort(vertex, nbVertices, sizeof(bspSortData), constructBSPTreeSort);

/*
The required translation of the base plane is now simply the
average of the signed distance of the two vertices that came out
in the middle after sorting:
*/
splitIdx = nbVertices / 2;
mv::Float d = 0.5 * (vertex[splitIdx - 1].signedDistance +
    vertex[splitIdx].signedDistance);

/*
We now translate the dual base plane, (un)dualize it and store
it in the pl of the BSP node:
*/
pl = dual(dualBasePlane - d * ni);

```

23.4 MODELING THE SCENE

This section treats the geometry used for modeling the scene. We need some way to place our light sources, cameras, and polygon models. So we need both methods to represent and apply Euclidean transformations, and methods to interactively manipulate these transformations.¹

In the conformal model, the natural choice to represent Euclidean transformations is to use rotors. In Chapter 16 we showed how conformal rotors can be used to represent and apply rotations, translations, reflections, and uniform scaling. These are the most basic transformations used in computer graphics, and they would classically be represented with 4×4 matrices.

To apply a conformal rotor to a blade, we use code like:

```
/*  
xf is the rotor (xf is short for 'transform').  
ptLocal is a point that we want to take from a  
'local' coordinate frame to the global coordinate frame.  
*/  
pt = xf * ptLocal * reverse(xf);
```

23.4.1 SCENE TRANSFORMATIONS

Concatenating Transformation Rotors

During interactive modeling, we want to concatenate small, interactively specified transformations to existing transformations. This can be done by either pre- or postmultiplying the existing rotor with the new rotor. As explained in Section 14.6.1, premultiplying applies the transformation in the global frame; postmultiplying applies the transformation in the frame represented by the current rotor. Which method is most practical depends on which frame gives the parameters that specify the small transformation.

Knowing how to apply rotors to blades and how to combine rotors, the only problem left is to construct the required transformation rotors in response to interactive user input. Figure 23.2 shows a screenshot of the user interface of the modeler. The caption of the figure lists all the available manipulations; most of them we treat below.

Mouse Input

All manipulations are done in response to mouse motions that arrive from the user interface toolkit as (x, y) coordinates. We immediately transfer these coordinates to a 2-D Euclidean vector:

¹ We will not treat interactive modeling of the polygonal models themselves (e.g., manipulating the individual vertices of a model), although the geometry required to do that is similar to the techniques exposed here.

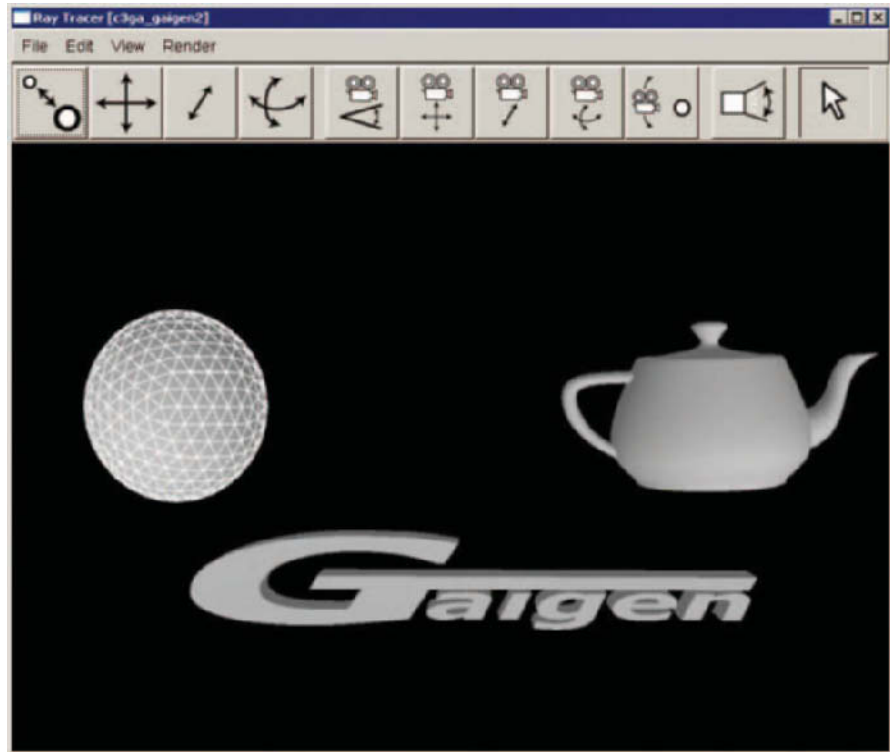


Figure 23.2: Screenshot of the user interface of the modeler. In the main viewport cameras, models, and light sources can be manipulated using the mouse. The available options are denoted by the buttons: scaling, translation (parallel to viewport), translation (orthogonal to viewport), rotation, camera field of view (active in this screenshot), camera translation (parallel to viewport), camera translation (orthogonal to viewport), camera rotation, camera orbit, size of spotlight, and selection. The selected model or light source is indicated by superimposing its wireframe mesh on top of its solid rendering.

```
vectorE2GA modelingWindow::getFLTKmousePosition() const {
    /*
     * The calls to Fl::event_x() and Fl::event_y() return the coordinates
     * of the mouse in pixels. We reflect the y-coordinate by subtracting
     * it from the height h() of the window.
     */
    return Fl::event_x() * e1 + (h() - Fl::event_y()) * e2;
}
```

By subtracting mouse positions from two consecutive mouse events, we get the `mouseMotion`, which is used in the code below.

Scaling an Object

The first transformation we treat is scaling. To scale an object (model or light source) in a way that is intuitive to the user, we need a scaling rotor at a specific location; that is, at the center of the model. By *post*multiplying by the current transform of the object, we can apply the scaling rotor at the location of the model with no extra effort. Therefore, it is enough to specify the scaling rotor at the origin:

```
scalar createScalingVensor(const vectorE2GA &mouseMotion) {
    /*
     * We use the 'vertical' part of the mouse motion to
     * indicate the amount of scaling:
     */
    return exp((0.01 * (e2 << mouseMotion) * (no ^ ni)));
}
```

Note that we extract the vertical part of the motion (i.e., dragging the mouse up and down will grow or shrink the object), and automatically uses a logarithmic scale (see Section 16.3.1).

Translating an Object

Translation is only slightly harder. The direction of the translation is specified in the camera frame though `mouseMotion`. We need to transform it into the global frame before we can *pre*multiply it with the object transform. Here is the code that computes a rotor for translation parallel to the camera:

```
/*
 * 'cameraXF' is the current transform of the camera.
 */
translator createTranslationVensorParallel(
    const TRSVensor &cameraXF,
    const vectorE2GA &scaledMouseMotionInCameraFrame) {
    /*
     * Compute translator in camera frame.
     * scaledMouseMotionInCameraFrame is
     * 'ready for use' except that it is in the wrong frame.
     *
     * So we compute the free vector that represents the
     * mouse motion, transform that to global coordinates,
     * and then compute the exponent of that:
     */
    return exp(0.5 * cameraXF *
        (ni ^ scaledMouseMotionInCameraFrame) *
        reverse(cameraXF));
}
```

Note that the mouse motion must be scaled such that the selected model appears to stick to the cursor. For this to work, the mouse motion is scaled according to the distance of the object to the camera and the field of view of the camera:

```

/*
getSelectionDepth() returns the distance of the selected
object to the camera.
C.getFOVwidth() returns the field of view of the camera at
distance 1 to the camera.
w() returns the width of the viewport.
*/
vectorE2GA scaledMouseMotionInCameraFrame =
    (mouseMotion * getSelectionDepth() * C.getFOVwidth() / w());

```

Object translation orthogonal to the camera is similar to parallel translation. One difference is that we scale the mouse motion according to the current distance of the object, so that the further away the object is, the faster it will translate in response to vertical mouse motion.

Rotating an Object

Rotation is the most involved object transformation. When the user selects the rotation mode, a transparent bounding sphere is drawn around the model (this is indeed the bounding sphere that we computed earlier). The user can grab any point on the sphere and drag it around. The object will follow the motion as indicated by the user.

Figure 23.3 shows how we compute the required rotation. First, we construct two lines into the scene and compute where they intersect the bounding sphere of the object. The lines go from camera position through the previous and current mouse positions, respectively.

When we have computed the intersection points, we construct two lines from the center of the bounding sphere through the two points. The rotor turning one into the other can be constructed from their halfway line, which is computed from the point average of the two conformal points. This is essentially the trick of (10.13) in the vector space model that we extended to the conformal model in Section 14.6.2.

Here is the code that computes the rotor based on the two intersection points:

```

sphere s = dual(getSelectedObjectBoundingSphere());

// Get the intersection points:
point ptCurrent = getRotatePoint();
point ptPrevious = getPreviousRotatePoint();

// Compute the two lines by wedging the points/spheres together:
line L1 = s ^ ptCurrent ^ ni;
line L2 = s ^ (ptCurrent + ptPrevious) ^ ni;

// Compute the (normalized) versor, and apply to object transform:
L.preMulXF(unit_r(L1 * L2));

```

The function `unit_r()` computes the unit line under the reverse norm (i.e., $L/\sqrt{-L \cdot \tilde{L}}$).

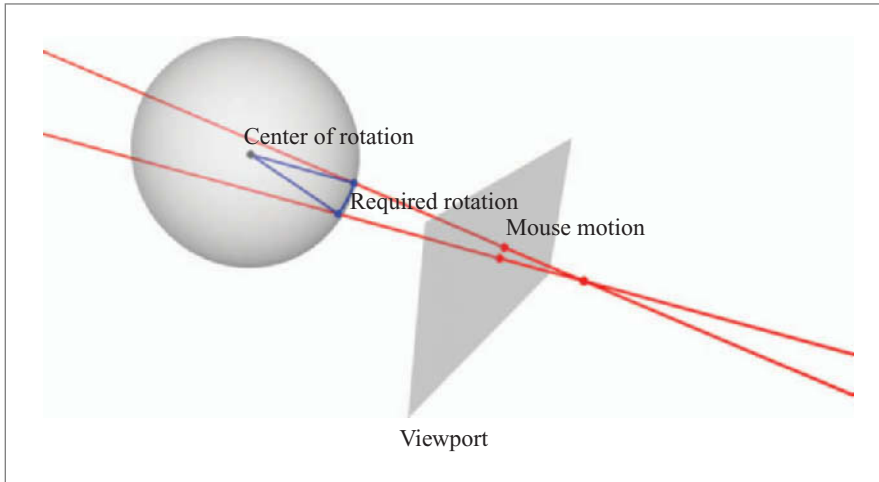


Figure 23.3: Rotating an object. On the right is the camera center (a red point). The user specifies the rotation by dragging the mouse in the viewport plane. This results in two points in the image plane (the current and previous mouse position). The two lines from the camera center through the mouse locations intersect with the bounding sphere in the blue points. The ratio of the lines from the center of the bounding sphere through both intersection points gives (twice) the required rotation.

Translating the Camera

Translating the camera is simpler than translating an object. Because the `mouseMotion` is specified in the camera frame, we can directly form a translation rotor:

```
TRSvector createCameraTranslationVersorParallel
(const vectorE2GA &mouseMotion) {
    // Exponentiate the free vector to get the translation versor:
    return exp((0.5 * (mouseMotion ^ ni)));
}
```

For the depth translation, we use only the vertical part of the mouse motion:

```
TRSvector createCameraTranslationVersorOrthogonal
(const vectorE2GA &mouseMotion)
{
    /*
    Same code as in create_translation_versor_parallel,
    except we replaced mouseMotion with
    ((e2 << mouseMotion) * e3):
    */
    vectorE3GA mouseMotionOrthogonalCameraPlane =
        (((e2 << mouseMotion) * e3) ^ ni);
}
```

```

/*
We can now directly exponentiate this free vector
to get our translation versor:
*/
return exp(0.5 * mouseMotionOrthogonalCameraPlane);
}

```

Rotating the Camera

Camera rotation is done through a spaceball interface (see Figure 23.4). If the user drags the mouse on the outside of the (invisible) circle, the camera rotates about the $\mathbf{e}_3$ vector: a rotation in the screen plane. If the user drags the mouse inside the dashed circle, the camera rotates about the local $\mathbf{e}_1$ and $\mathbf{e}_2$ vectors.

Construction of the rotation rotors is satisfyingly straightforward. For a rotation in the screen plane, we compute the 2-blade spanned by the mouse motion and mouse position:

```

TRSversor createCameraRotationVersorInScreenPlane(
    const vectorE2GA &mousePosition,
    const vectorE2GA &mouseMotion)
{
    return exp(mouseMotion ^ mousePosition);
}

```

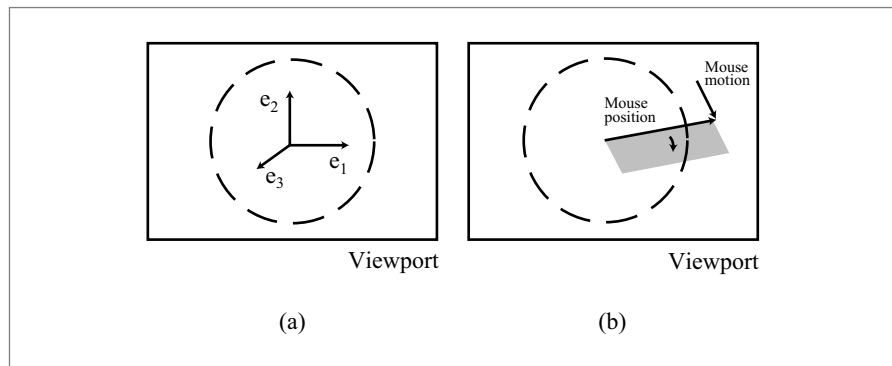


Figure 23.4: The spaceball interface. (a) When the user drags the mouse on the inside of the dashed circle the camera rotates about $\mathbf{e}_1$ and $\mathbf{e}_2$; outside of the circle, the camera rotates about $\mathbf{e}_3$. (b) Computing the rotation when the user drags the mouse on the outside of the circle. We compute the 2-blade (drawn in gray) spanned by the mouse motion and the mouse position (drawn as vectors). The exponential of that 2-blade is the required rotation rotor.

For a rotation outside the screen plane, we compute exponential of the 2-blade $e_3 \wedge \text{mouseMotion}$:

```

    versor createCameraRotationVersorOutsideScreenPlane(
        const vectorE2GA &mouseMotion)
    {
        return exp(e3 ^ mouseMotion);
    }

```

Orbiting the Camera around a Selected Object

Orbiting the camera around a selected object is the most complicated transformation. The difficulty is that we want to rotate about a point in the world frame (the center of the selected object), with the rotation angle and attitude specified in the camera frame.

The standard way to construct a rotation about an axis through an arbitrary point is to first create a rotation rotor about the origin, and then to translate that rotor. So, first we compute a translator for the offset from the camera to the rotation rotor we want to use for orbiting. We do this by transforming the flat point at the origin according to the orbit rotor and transforming that point into the camera frame:

```

// Get the offset from the camera to the orbit versor.
// C.getXF() returns the camera transform.
// getOrbitVersor() returns the transform of the object we want
// to orbit
flatPoint t =
    reverse(C.getXF()) * getOrbitVersor() *
    (no ^ ni) *
    reverse(getOrbitVersor()) * C.getXF();

```

Once we have the location of the orbit rotor relative to the camera frame as a normalized flat point, we create a translator to that location:

```

// compute the translation versor from the origin to 't'
translator T = exp(- 0.5 * (t - (no ^ ni)));

```

Note that we cannot apply `reverse(C.getXF()) * getOrbitVersor()` directly to our translator T , because it may contain unwanted scaling and rotation; hence the trick of transforming a flat origin.

The actual rotation is computed from the mouse parameters in the same way as with a regular camera rotation (see above), but it now needs to be performed at the location specified by the translation rotor T :

```

// post multiply the camera transform with the rotation 'xf'
C.postMulXF(T * xf * reverse(T));

```


23.5 TRACING THE RAYS

With our scenes (interactively) modeled and represented, we arrive at the core of the ray tracer. In this section we discuss how rays are represented, how they are spawned, and how the ray tracer computes the intersection of rays with the models.

23.5.1 THE REPRESENTATION OF RAYS

Since rays interact with almost every part of our ray tracer, we should choose their representation carefully. We consider several alternatives and pick the one that we think works best in the context of the operations that need to be done.

Classical Representation

A ray is a directed half-line, with a position (where it was spawned) and a direction. Classically, we would represent this as the point (requiring three or four coordinates) and a direction vector (requiring three coordinates).

Point-Line Representation

A sensible generalization of this classical representation in CGA would be to use a conformal point and a conformal line through it. This is, of course, somewhat redundant, since the line also contains partial position information. But it would seem convenient to have the ray in line representation so that it can be used directly in intersection testing.

However, this representation requires a lot of coordinates for storage (five for the point, six for the line). A regular conformal point might seem a good way to represent position, but it is awkward in a ray tracer. When we intersect the ray (line) with a plane (the most common intersection operation in the ray tracer), we get a flat point as a result. To compute the representation of a spawned ray at that location, that flat point would have to be converted to a regular point, which is extra work not warranted by the geometry of the situation. Also, the point/line combination is expensive to transform to and from coordinate frames. Finally, even though it is algebraically possible to reflect and refract the lines by substituting them in the classical equations for reflection and refraction of directions (illustrated for reflection in Section 13.4), this is rather computationally expensive.

Tangent Vector Representation

A third idea is to use a tangent vector to represent rays. It seems perfect: a tangent vector has position and direction all in one. This would represent a ray as a single blade, conforming to the geometric algebra philosophy of representing one object by one blade, whenever possible. Besides, a tangent vector can be turned into its carrier line at any time (just add infinity using the outer product).

Yet this representation suffers from the same problems as the point-line representation. A tangent vector requires 10 coordinates for storage, and this makes it expensive in

operations. Moreover, extraction of the locational part of a tangent vector cannot be done by a computationally free coordinate transfer (this problem was already described for tangent bivectors in Section 23.3).

Rotor Representation

Another possible ray representation is as a rotor, since a translation/rotation rotor naturally represents a position and a direction (the axis of a general rotation). It requires just seven coordinates for storage. But the position and direction information cannot be extracted for free from a rotor V by coordinate extraction. Instead, we have to apply the rotor to some blade to determine such properties: $V o V^{-1}$ is the position of the ray (as a conformal point), and $V(\mathbf{e}_3 \wedge \infty) V^{-1}$ is the direction of the ray. Both these equations are relatively expensive to evaluate.

More seriously, we need to intersect rays, but there is no `meet` operation for rotors (since they are not blades).

Flat Point-Free Vector Representation

In the end, we settled for the less elegant but more efficient representation of a ray as a flat point with a free vector. The flat point gives the position, the free vector gives the direction. The choice of a flat point may not seem natural. You cannot directly create a line from a flat point $p \wedge \infty$ and a free vector $\mathbf{u} \wedge \infty$, since they both contain an infinity factor. This is solved by removing the infinity factor from the free vector before wedging them together:

```
// fp is a flat point, fv is a free vector
line l = fp ^ (- no << fv)
```

The number of coordinates of a (flat point, free vector)-pair is as low as the classical representation. But, unlike the classical representation, the flat point and free vector automatically have clear semantics in operations. The flat point responds to both translation and rotation, the free vector only to rotation (it is translationally invariant).

Still, the disadvantage of this mixed CGA representation is that we miss out on automatic algebraic properties for the ray as a whole; the programmer has to remember that the flat point and the free vector belong together and what their combination means. Had we used a tangent vector, there would be no possibility of getting its raylike properties wrong in transformations. Now the programmer must specify semantics that are not intrinsic to the algebra by putting the blades together in one class and adding comments that give detailed information on what each blade represents. For instance:

```
class ray {
public:
    flatPoint pos;           // position of the ray
    freeVector direction;    // direction of the ray
};
```

Another downside of the (flat point, free vector)-pair representation is that distance computations become less elegant. In ray tracing, when we search along the ray for the first intersection we need to compute the distance of candidate intersections with the start of the ray. If we had the ray position as a regular conformal point, we could do that with a simple inner product. With the ray position as a flat point, we have to use the classical approach of subtracting the two points and computing the Euclidean norm (squared).

23.5.2 SPAWNING RAYS

There are four occasions where we have to spawn new rays:

- The initial spawning of a ray through camera center and image plane;
- For shadow rays that test whether a light source is visible from a specific point;
- When reflecting a ray;
- When refracting a ray.

The geometry involved in doing reflection and refractions is treated in separate sections. Here we just show how camera rays and shadow rays are spawned.

Camera Rays

To render an image, we trace rays through each pixel in the image. If antialiasing is required, we spawn multiple rays through each pixel, each ray offset by a small random vector. The next block of code spawns the initial rays:

```
mv::Float pixelWidth = cameraFOVwidth / imageWidth;
mv::Float pixelHeight = cameraFOVheight / imageHeight;

// for each pixel in the image:
for (int y = 0; y < imageHeight; y++) {
    for (int x = 0; x < imageWidth; x++) {
        /*
         * Compute the direction of the ray if it has to
         * got to go through sensor pixel [x, y]:
         */
        freeVector rayDirection =
            ((x * pixelWidth - 0.5 * cameraFOVwidth) * e1 +
             (y * pixelHeight - 0.5 * cameraFOVheight) * e2 +
             e3) ^ ni;

        // sample multiple times, accumulate the result in pix_color:
        color pixelColor;
        for (int s = 0; s < multisample; s++){
```

```

/*
Add a small perturbation within the square.
To generate random numbers, we call mt_rand(),
the Mersenne twister random number generator.
*/
freeVector perturbedRayDirection =
    rayDirection +
    ((mt_rand() - 0.5) * e1 * pixelWidth
    (mt_rand() - 0.5) * e2 * pixelHeight) ^ ni);

// make the direction unit:
freeVector unitRayDirection = unit_e(perturbedRayDirection);

// trace the ray from camera position towards 'unitRayDirection'
ray R((C.getPosition(), unitRayDirection);
pixelColor += trace(R, maxRecursion);
}
}

```

Note that one could say that we make direct use of (image) coordinates to specify the directions of the rays ($x\mathbf{e}_1, y\mathbf{e}_2$), which is somewhat against our philosophy, but we believe this is the most sensible way to start the rays. Trying to avoid coordinate usage here would make things more awkward instead of simpler.

Shadow Rays

Shadow rays are spawned to check whether there is a clear path between some surface point and a light source. The surface point is naturally represented as a flat point, since it comes from an intersection computation.

The light source position is encoded in a rotor. We can extract a point from that rotor by using it to transform the flat point at the origin:

```

flatPoint lightPoint =
    light->getXF() * (no ^ ni) * reverse(light->getXF());

```

This computation can be done before we start rendering the scene, since the position of the light does not change during the rendering of a single frame.

We now have two flat points that we can simply subtract to get a free vector that points from the surface point to the light:

```

/*
'surfacePoint' is a flat point at the surface of some model.
'lightPoint' is a flat point at the position of the light for
which we have to check visibility:

We can subtract the points since they are both normalized.
*/

```

```
freeVector shadowRayDirection =
    unit_e(lightPoint - surfacePoint);
```

This gets us the representation of the shadow ray as `surfacePoint` and `shadowRayDirection`.

23.5.3 RAY-MODEL INTERSECTION

A typical ray tracer spends most of its time finding if and where rays intersect models. Our ray tracer does this in two steps. First, it checks if the ray intersects the bounding sphere. If so, a descent down the BSP tree follows, until it eventually finds actual intersections with faces of the model.

Bounding Sphere Test

The bounding sphere computation is quite simple:

```
// compute the line representation of the ray:
line rayLine = ray.pos ^ (- no << ray.direction);

// intersect the line and the sphere
pointPair intersection = dual(rayLine) << boundingSphere;

/*
Check if intersection is a real circle by computing
the radius squared:
*/
if ((intersection << intersection) > 0) {
    /*
    intersection with bounding sphere detected:
    . . .
    */
}
```

A Trip Down the BSP Tree

The descent down the BSP tree is more involved. Remember that the BSP tree recursively splits space into halves. The leaves of the tree contain the model faces that lie in that particular partition of space. During an intersection test, the goal of the BSP tree is to quickly arrive in those faces (leaves of the BSP tree) that the ray actually intersects. For instance, the first partition plane of the BSP tree divides space into two halves. Unless the ray happens to be parallel to this plane, it always intersects both halves, so we split the line in two, and each line segment goes down to the appropriate half of the BSP tree to be tested for intersections. Of course, we can take advantage of the fact that no faces lie outside of the bounding sphere: before we start our descent, we clip the ray against the bounding sphere, resulting in the initial line segment.

On the geometric algebra side of things, what this all boils down to is that we require a representation for line segments. This specific representation must be picked so that it works well while descending down a BSP tree.

We decided to use a pair of flat points to represent such line segments. One flat point represents the start and the other the end of the line segment. We now show how the BSP descent algorithm looks with this choice of representation. Afterwards, we briefly discuss the alternative representation of CGA point pairs to represent line segments.

To bootstrap the BSP descent algorithm, we need to have a line segment that represents the ray as clipped against the bounding sphere. As input for the recursive descent, we take the point pair named `intersection` from the previous code fragment and dissect that into two flat points using (14.13):

```
/*
  Dissects a point pair 'pp' into two flat points 'fpt1' and 'fpt2'
*/
void dissectPointPair(const pointPair &pp,
  normalizedFlatPoint &fpt1, normalizedFlatPoint &fpt2) {
  mv::Float n = sqrt(pp << pp);
  dualPlane v1 = ni << pp;
  dualPlane v2 = n * v1;
  dualSphere v3 = v1 << pp;

  mv::Float scale = 1.0 / (v3 << ni);
  fpt1 = (scale * ni) ^ (v3 + v2);
  fpt2 = (scale * ni) ^ (v3 - v2);
}
```

We sort the two points according to distance from the start of the ray. This is useful because it allows us to prune part of the BSP tree descent: we want to find the closest intersection of the ray with the model. So we first check the parts of the BSP tree closest to the start of the ray to see if we find any intersection there.

We determine what part of the tree to descend in first by checking on what side of the plane the start of the ray lies:

```
// partitionPlane is the plane that splits the space in two halves.
if ((ray.pos << dual(partitionPlane)) > 0.0) {
  // descend front side first
}
else if ((ray.pos << dual(partitionPlane)) < 0.0) {
  // descend back side first
}
else {
  // _always_ descend both sides
}
```

For the actual descent of the tree, we first check on what side(s) of the partition plane our line segment lies (this is not necessarily the same side as the start of the ray):

```

/*
partitionPlane is the plane that splits the space in two halves.
We compute the signed distance of the two flat points to the plane:
*/
mv::Float sd1 = no << (dual(partitionPlane) << fp1);
mv::Float sd2 = no << (dual(partitionPlane) << fp2);

```

If the line segment lies on both sides of the partition plane, we have to compute the intersection point of the line segment and the plane. Otherwise, we can simply descend down the appropriate node in the BSP tree.

The following code checks if we have to compute the intersection point, and computes it:

```

/*
If one signed distance is negative, and the other is positive,
then the line segment must lie on both sides of the plane:
*/
if (sd1 * sd2 < 0.0) {
    /*
    Compute the intersection point:
    -compute the line representation of the ray
    -intersect rayLine line and the partitionPlane
    */
    line rayLine = ray.pos ^ (- no << ray.direction);
    flatPoint fpI = unit_r(dual(partitionPlane) << line);

    // descend further down both sides of the BSP tree
    // . . .
}

```

A degenerate case occurs when the line segment lies inside the partition plane. We then simply descend to both children of the current BSP node.

Representing Line Segments with Point Pairs

Instead of pairs of flat points, we could use point pairs (i.e., 0-spheres) to represent line segments. But at some point we will have to dissect the point pair into two points to form two new point pairs (each on a different side of the partition plane), and this would then require the rather expensive (14.13). It is computationally more efficient to have the ends of the segments readily available.

23.5.4 REFLECTION

Reflections are computed as part of the shading computation treated in the next section. Reflection of any blade in any other blade is quite simple in the conformal model, though there are some signs depending on whether they are represented directly or dually (see Table 7.1). The following function implements the reflection. It forms a plane from the surface attitude (a free bivector), and reflects the ray direction (a free vector) in it:

```

color scene::reflect(const ray &R, int maxRecursion, const
                    surfacePoint &spt) const {

```

```

/*
Reflect the ray direction in the surface attitude,
and spawn a new ray with that direction at the surface point.
*/
ray reflectedRay(
    spt.getPt(),                // starting position of new ray
    -((spt.getAtt() ^ no) *
    R.get_direction() *
    reverse(spt.getAtt() ^ no)) // direction of new ray
);

return trace(reflectedRay, maxRecursion - 1);
}

```

23.5.5 REFRACTION

Like reflections, refractions are also computed as part of the shading computation. The classical equation for refraction of direction vectors is:

$$\mathbf{u}' = \left(\text{sign}(\mathbf{n} \cdot \mathbf{u}) \sqrt{1 - \eta^2 + (\mathbf{n} \cdot \mathbf{u})^2 \eta^2} - (\mathbf{n} \cdot \mathbf{u}) \eta \right) \mathbf{n} + \eta \mathbf{u} \quad (23.1)$$

where $\mathbf{n}$ is the surface normal, $\mathbf{u}$ is the ray direction, and η is the refractive constant. In our conformal implementation, both $\mathbf{n}$ and $\mathbf{u}$ are free vectors that can be plugged directly into the equation. As an interesting side note we mention that the same equation holds at the level of intersecting lines, by the extension principles of Section 13.4.

23.6 SHADING

The ray tracer performs shading computations for every spawned ray that intersects a model (unless it is a shadow ray). The shading computations should approximate how much light the material reflects along direction of the ray (and thus towards the camera). We do not discuss the shading computations in detail because they do not differ much from the classical computations. Even in the conformal model, shading computations are most conveniently performed using 3-D Euclidean vectors. For completeness, we briefly list the steps involved in shading:

- Before the ray tracer can perform any shading computations, it must interpolate the surface properties defined at the vertices for the intersection point. This is done by barycentric interpolation (see (11.19))
- If the model has a bump map applied to it, we have to modulate the surface attitude according to that bump map. This again involves interpolation.

- Finally, we have to perform the actual shading computations. Our ray tracer mimics the simple fixed function pipeline shading model of OpenGL, although we perform shading per intersection point instead of per vertex.

23.7 EVALUATION

We have shown most of the geometry involved in implementing a classical ray tracer. We have emphasized that there are usually several ways to represent computer graphics primitives in the conformal model of Euclidean geometry. For the ray tracer, we picked the most efficient representations we could find, but in other, less time-sensitive applications, we may pick the more elegant universal representations from the conformal model, since we do think it is ultimately beneficial represent each geometric concept with a single type of blade. It would permit one to build the elements of a geometry library independent of the particular application they originated in.

Most interactive modeling transformations have been reduced to simple one-liners, direct conformal model formulas involving the mouse parameters. This is a part of the code where the conformal model shows off some of its power. The directness of the approach helped make the actual writing of the code a quick and enjoyable exercise.

The direct use of coordinates in the equations and the source code has been eliminated, except for places where they were actually useful (e.g., systematically generating position of all pixels in a 2-D image). This would not be possible with classical homogeneous coordinates. Just pick up any computer graphics text that treats construction of Euclidean transformations, and the pages will be littered with 4×4 matrices filled with coordinates.

As we already saw in the benchmarks in Chapter 22, there is a performance penalty for using the 5-D conformal model over the 3-D vector space model: our conformal ray tracer is about 25 percent slower than the vector space version. The difference compared to the 4-D homogeneous version is less than 5 percent.

In the graphical user interface of Section 23.4, we demonstrated the most pure manifestation of the conformal model. There, the correspondence to the algebraic elegance of the model versors and the simplicity of the code was most striking. We view it as an ideal example of what can be achieved by the methods for object-oriented programming of geometry exposed in this book.

This Page Intentionally Left Blank